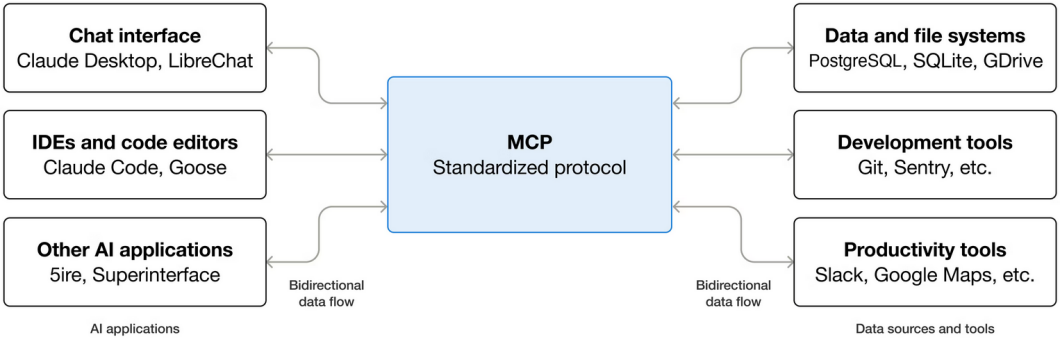
Simple MCP Tool Server

A quick hands-on for creating an MCP tool server

C.D. Milner

March 2026

What is MCP?



The Model Context Protocol (MCP) is a standardized protocol for connecting AI applications to external systems, such as data sources and tools.

Source: https://modelcontextprotocol.io/docs/getting-started/intro

The MCP Server

The MCP server is what we use to provide tools and data sources to our models.

For this lesson, we will only look at implementing tools and not look at how to implement data sources.

For information on connecting to data sources, see: https://modelcontextprotocol.io/docs/develop/build-server

mcp_lesson_server.py

```python
from fastmcp import FastMCP

mcp = FastMCP("MCP Lesson Server")

@mcp.tool
def some_tool():
    """A description of some tool"""
    [Some tool code]
    return

if __name__ == "__main__":
    mcp.run()
```

The MCP Client

The MCP client is what ties together our model execution and our MCP server.

Some applications, like Claude, have this built in. We will be using a simple python script with ollama and FastMCP as our client.

For information on building custom clients, see: https://modelcontextprotocol.io/docs/develop/build-client

mcp_client_chatbot.py

```python
import ollama, asyncio
from fastmcp import Client

client = Client("http://localhost:8050/mcp")

async def main(model: str):
    [Some setup code]
    while True:
        [Some chatbot code]

if __name__ == "__main__":
    asyncio.run(main(model='ministral-3'))
```

Simple Tool Docstring Guide

mcp_lesson_server.py

```
...
@mcp.tool
def some_tool(some_var: vartype):
    """A short but concise description of the tool.
    Args:
        some_var (vartype): This is some variable required for the tool.
    Example:
        Give examples or edge cases to help the model use the tool.
    Returns:
        somevar: Describe the output here.
    Note:
        A catchall I personally use if I can't fit it anywhere else."""
...
```

Simple Self Contained Tools

mcp_lesson_server.py

```python
...
@mcp.tool
def hard_question():
    """Asks a hard question.
    Returns:
        string: A string with a hard question.
    Note:
        If the user ever asks for a hard question, this is it."""

    answer = "Are you telling me a shrimp fried this rice?"

    return answer
...
```

Tools That Call External Programs

mcp_lesson_server.py

```
...
@mcp.tool
def get_date():
    """Returns the current date and time.
    Returns:
        string: A string with the current date and time"""

    current_date = subprocess.run(["date"],
                                  capture_output=True,
                                  text=True,
                                  check=True)
    return current_date
...
```